

Рекурсия

Рекурсия в Си - это техника программирования, при которой функция вызывает саму себя во время выполнения.

Рекурсивные функции подходят для задач, которые естественным образом делятся на похожие подзадачи, например, обход структур данных (деревьев, связанных списков), вычисление факториала и тд.

Рекурсия может быть прямой (функция вызывает саму себя) или косвенной (когда одна функция вызывает другую, которая затем вызывает первую).

Рекурсия часто менее эффективна по памяти чем циклы.

Для хранения порядка вызова рекурсивной функции используется стек вызовов, работающий по принципу Last In – First Out.

- 1) Указатели на функцию
- 2) Требования рекурсии
- 3) Отладка
- 4) Примеры

1. Указатели на функцию

В Си функция имеет адрес и может иметь указатель. Указатель на функцию представляет собой выражение или переменную, которые используются для представления адреса функции. Указатель на функцию содержит адрес начала области памяти, в которой располагается выполняемый код функции.

Самым распространенным указателем на функцию является ее имя. С помощью имени функции мы можем вызывать ее и получать результат ее работы.

Но также указатель на функцию можно определять в виде отдельной переменной с помощью следующего синтаксиса:

```
тип (*имя_указателя) (типы_параметров);
```

Где:

тип - тип возвращаемого функцией значения.

имя_указателя - произвольно выбранный идентификатор в соответствии с правилами о наименовании переменных.

После названия указателя в скобках идут через запятую типы параметров. Если функция не принимает параметров, то указывается `void`.

При определении указателя стоит обратить внимание на скобки вокруг имени. Например, определение

```
void (*message) (void);
```

НЕ будет аналогично следующему определению:

```
void *message (void);
```

Во втором случае определен не указатель на функцию, а прототип функции `message`, которая возвращает указатель типа `void*`.

Следует учитывать, что указатель на функцию должен по типу возвращаемого значения и типу параметров соответствовать функции, иначе он не сможет соответствовать этой функции.

Например, определим указатель на функцию:

```
void (*message) (void);
```

и применим этот указатель на функции:

```
#include <stdio.h>

void hello()
{
    printf("Hello, World \n");
}

void goodbye()
{
    printf("Good Bye, World \n");
}
```

```

}

int main(void)
{
    // определяем указатель на функцию
    void (*message) (void);

    message = hello; // указатель указывает на функцию hello
    message();        // вызываем функцию, на которую указывает
    указатель

    message = goodbye; // указатель указывает на функцию
    goodbye
    message();        // вызываем функцию, на которую указывает
    указатель

    return 0;
}

```

Указателю на функцию можно присвоить функцию, которая соответствует указателю по возвращаемому типу и спецификации параметров:

```
message = hello;
```

То есть в данном случае указатель `message` теперь хранит адрес функции `hello`. И посредством обращения к указателю мы можем вызвать эту функцию:

```
message();
```

Впоследствии мы можем присвоить указателю адрес другой функции, как в данном случае:

```
message = goodbye;
```

Вывод программы:

Hello, World

Good Bye, World

Рассмотрим еще один указатель на функцию, которая возвращает значение типа `int` и принимает два параметра типа `int`:

```
#include <stdio.h>

int sum(int x, int y)
{
    return x + y;
}

int subtract(int x, int y)
{
    return x - y;
}

int main(void)
{
    int a = 10;
    int b = 5;
    int result;
    int (*operation)(int, int);
```

```

    operation = sum;
    result = operation(a, b);
    printf("result = %d \n", result);          // result=15

    operation = subtract;
    result = operation(a, b);
    printf("result = %d \n", result);          // result=5

    return 0;
}

```

Вывод:

result = 15

result = 5

Здесь определен указатель `operation`, который может указывать на функцию с двумя параметрами типа `int`, возвращающую также значение типа `int`. Соответственно мы можем присвоить указателю адреса функций `sum` и `subtract` и вызвать их, передав при вызове в указатель нужные значения для параметров.

Массивы указателей на функции

Мы можем определять массивы указателей на функции. Для этого используется следующий формальный синтаксис:

тип (*имя_массива[размер]) (параметры)

Например:

```
double (*actions[]) (int, int)
```

Здесь `actions` представляет массив указателей на функции, каждая из которых обязательно должна принимать два параметра типа `int` и возвращать значение типа `double`.

Посмотрим применение массива указателей на функции на примере:

```
#include <stdio.h>

void sum(int x, int y)
{
    printf("x + y = %d \n", x + y);
}

void subtract(int x, int y)
{
    printf("x - y = %d \n", x - y);
}

void multiply(int x, int y)
{
    printf("x * y = %d \n", x * y);
}

int main(void)
{
    int a = 10;
    int b = 5;
    void (*operations[3])(int, int) = {sum, subtract, multiply};
    // получаем длину массива
    int length = sizeof(operations) / sizeof(operations[0]);
```

```
for (int i = 0; i < length; i++)
{
    operations[i](a, b);    // вызов функции по указателю
}

return 0;

}
```

Вывод:

$x + y = 15$

$x - y = 5$

$x * y = 50$

Здесь массив `operations` содержит три функции `sum`, `subtract` и `multiply`, которые последовательно вызываются в цикле через перебор массива в функции `main`.

2. Требования рекурсии

- Базовый случай (условие остановки). Обязательно нужно определить условие, при котором рекурсия прекратится. Без базового случая функция будет вызывать себя бесконечно, что приведёт к переполнению стека.
- Рекурсивный шаг (приближение к базовому случаю). Функция должна вызывать себя с изменёнными входными данными, которые постепенно приближают выполнение к базовому случаю. Например, при вычислении чисел

Фибоначчи каждое последующее число является суммой двух предыдущих.

- Ограничение глубины рекурсии. Количество рекурсивных вызовов ограничено размером стека. При каждом вызове функции для параметров и переменных (`auto` и `register`) выделяется новая память, чтобы не перезаписывались их значения из предыдущих вызовов. Это может привести к переполнению стека при глубокой рекурсии.
- Учёт особенностей переменных с описателем `static`. При каждом вызове функции для локальных переменных выделяется отдельная область памяти. Это означает, что локальные переменные в разных вызовах функции не влияют друг на друга. Для переменных, объявленных с описателем `static`, новая память при новом рекурсивном вызове не выделяется – их память существует в течение всего времени жизни программы.
- Использование хвостовой* рекурсии для оптимизации (опционально).

* Хвостовая рекурсия – это особый вид рекурсии, при котором рекурсивный вызов является последней операцией в функции перед возвратом результата. Это позволяет оптимизировать использование стека, так как компиляторы могут (хотя это не гарантировано) преобразовывать хвостовую рекурсию в итерацию, устраняя накладные расходы на сохранение стековых кадров.

3. Отладка рекурсии

Общие методы отслеживания:

- Ручные таблицы отслеживания (отслеживание выполнения шаг за шагом)

- Отладочная печать
- Пошаговое выполнение отладчика
- Визуализация рекурсивных вызовов
- Отладка через расширенные инструменты отслеживания

```
#include <stdio.h>

int factorial(int n) {
    // Базовый случай
    if (n <= 1) {
        return 1;
    }

    // Рекурсивный случай
    return n * factorial(n - 1);
}

// Пример отслеживания для factorial(4)
int main() {
    int result = factorial(4);
    printf("result = %d", result);
    return 0;
}
```

Визуализация стека рекурсивных вызовов:

```
A[factorial(4)] --> B[factorial(3)]  
B --> C[factorial(2)]  
C --> D[factorial(1)]  
D --> E[Базовый случай достигнут]
```

4. Примеры

Факториал числа n:

```
#include <stdio.h>  
  
int factorial(int n)  
{  
    if (n <= 1)  
    {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}  
  
int main() {  
    int res = factorial(5);  
  
    printf("%d", res);  
    return 0;
```

```
}
```

Вывод: 120

Числа Фибоначчи:

```
#include <stdio.h>
```

```
int Fibonacci(int n)
```

```
{
```

```
    if (n < 30) {
```

```
        if (n <= 1) return n;
```

```
        return Fibonacci(n - 1) + Fibonacci(n - 2);
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main()
```

```
{
```

```
    int fib4 = Fibonacci(4);
```

```
    int fib5 = Fibonacci(5);
```

```
    int fib6 = Fibonacci(6);
```

```
    printf("4 Fibonacci number: %d \n", fib4);
```

```
    printf("5 Fibonacci number: %d \n", fib5);
```

```
    printf("6 Fibonacci number: %d \n", fib6);
```

```
        return 0;
    }
}
```

Вывод:

4 Fibonacci number: 3

5 Fibonacci number: 5

6 Fibonacci number: 8

Сумма натуральных чисел от 1 до k:

```
#include <stdio.h>

int sum(int k) {
    //printf("%d",k); //Как пример отладки
    if (k > 0) {
        return k + sum(k - 1);
    }
    else {
        return 0;
    }
}

int main() {
    int summ = sum(9);

    printf("%d", summ);
}
```

Вывод: 45

Косвенная рекурсия:

```
#include <stdio.h>

void funcB(int n);

void funcA(int n) {
    if (n > 0) {
        printf("%d ", n);
        funcB(n - 1);
    }
}

void funcB(int n) {
    if (n > 0) {
        printf("%d ", n);
        funcA(n / 2);
    }
}

int main() {
    funcA(10);
    return 0;
}
```

Вывод: 10 9 4 3 1

Хвостовая рекурсия:

```
#include <stdio.h>

int factorial(int n, int acc) {
    if (n == 0) {
        return acc; // Базовый случай: возвращаем
        накопленный результат
    }
    return factorial(n - 1, acc * n); // Рекурсивный
    вызов – последняя операция
}

int main() {
    int a = factorial(5, 1);
    printf("%d", a);
    return 0;
}
```

Вывод: 120